ECOMMERCE-DEMO · USING-DOCKER VS USING-KUBERNETES

Where the observability configuration lives: Docker vs Kubernetes

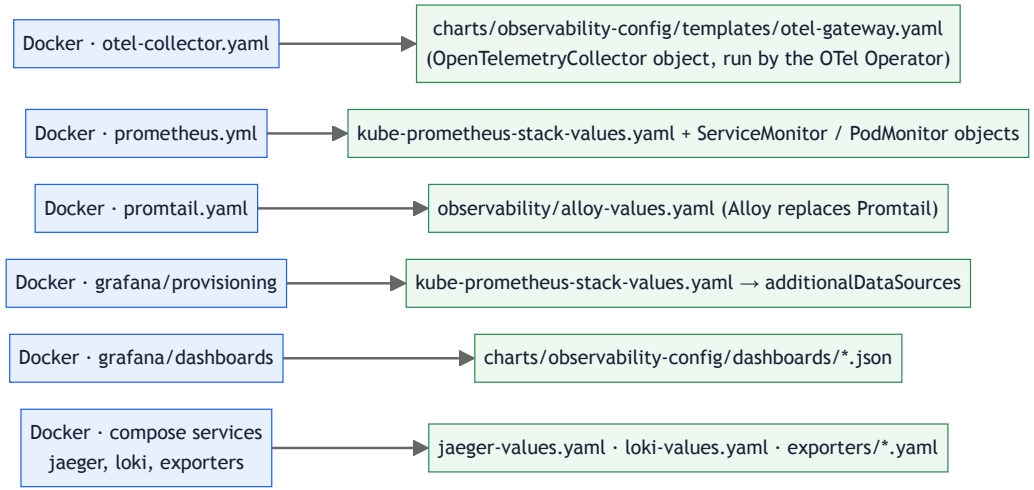
Short answer: Kubernetes still has a collector config — it just isn't in `using-kubernetes/observability/`. That folder only holds **settings for other people's Helm charts**. Our own configuration, including the collector, is a Helm chart: `using-kubernetes/charts/observability-config/`. The collector is in `templates/otel-gateway.yaml`.

Docker — using-docker/observability/

```
observability/
├── otel-collector.yaml      collector config (mounted into the container)
├── prometheus.yml           Prometheus scrape jobs
├── promtail.yaml            log shipping to Loki
└── grafana/
    ├── provisioning/        data sources + dashboard provider
    └── dashboards/          services-red.json, infrastructure.json
(+ jaeger, loki, exporters as services in docker-compose.yml)
```

Kubernetes — using-kubernetes/

```
observability/                          values for THIRD-PARTY charts
├── kube-prometheus-stack-values.yaml   Prometheus, Grafana (+ data sources), Alertmanager
├── otel-operator-values.yaml           the Operator that runs the collector
├── jaeger-values.yaml · loki-values.yaml
├── alloy-values.yaml                   log shipping (replaces Promtail)
└── exporters/                          postgres, redis, mongodb
charts/observability-config/            OUR chart (release obs-config)
├── templates/otel-gateway.yaml         ← the collector (Docker's otel-collector.yaml)
├── templates/middleware-monitoring.yaml  RabbitMQ + Traefik scrape targets
├── templates/dashboards.yaml + dashboards/*.json
├── templates/routes.yaml · rules.yaml
└── values.yaml                         switches: newrelic, alerts, routes
```



File-by-file mapping

Each Docker file has a Kubernetes counterpart. The big change: instead of mounting config files into containers, Kubernetes uses Helm values and **custom objects that operators turn into running config**.

DOCKER	KUBERNETES	WHAT CHANGED AND WHY
`otel-collector.yaml`	`charts/observability-config/templates/otel-gateway.yaml`	Same receivers / processors / exporters, wrapped in an **OpenTelemetryCollector** object. The **OTel Operator** creates the collector's Deployment, Service and ConfigMap from it, and rolls a new pod whenever it changes. Helm switches (`newrelic.enabled`) add or remove parts.
`prometheus.yml`	`observability/kube-prometheus-stack-values.yaml` (Prometheus settings) + **ServiceMonitor / PodMonitor** objects: `templates/middleware-monitoring.yaml` (RabbitMQ, Traefik), bottom of `otel-gateway.yaml` (collector), `serviceMonitor.enabled: true` in `observability/exporters/*.yaml`	No hand-written scrape list. The **Prometheus Operator** builds the scrape config from monitor objects found in any namespace, so new targets need no Prometheus edit.
`promtail.yaml`	`observability/alloy-values.yaml`	**Alloy replaces Promtail** (deprecated). Runs on every node as a DaemonSet; also masks PII, joins Java stack traces, extracts `trace_id` and log level.
`grafana/provisioning/datasources`	`additionalDataSources` in `observability/kube-prometheus-stack-values.yaml`	Grafana is part of kube-prometheus-stack; data sources (Loki, Jaeger with trace ↔ log ↔ metric links) are chart values.
`grafana/dashboards/*.json`	`charts/observability-config/dashboards/*.json` + `templates/dashboards.yaml`	Each JSON becomes a ConfigMap labelled `grafana_dashboard=1` ; Grafana's sidecar container loads it into the **Shop** folder.
jaeger, loki, exporters in `docker-compose.yml`	`observability/jaeger-values.yaml` , `loki-values.yaml` , `exporters/postgres|redis|mongodb-values.yaml`	Each is its own Helm release with pinned chart version (Jaeger 2.20, Loki single-binary, exporters with ServiceMonitors).
(none)	`observability/otel-operator-values.yaml`	New in Kubernetes: installs the Operator that runs the collector (uses the **contrib** collector image 0.160.0).
(none)	`charts/observability-config/templates/routes.yaml` , `rules.yaml`	Gateway API routes (grafana / prometheus / alertmanager .localhost) and alert rules + SLO (paused) — Docker published ports instead.

The collector side by side

The `config:` block of the Kubernetes object is the same language as Docker's file. Yellow = equivalent parts; green = new in Kubernetes.

Docker — otel-collector.yaml (mounted file)

```
receivers:
  otlp:
    protocols: { grpc: 0.0.0.0:4317, http: 0.0.0.0:4318 }
processors:
  memory_limiter: { limit_mib: 400, spike_limit_mib: 100 }
  batch: { timeout: 5s }
connectors:
  spanmetrics:            # RED metrics from spans
    exemplars: { enabled: true }
exporters:
  otlp/jaeger:  { endpoint: jaeger:4317 }
  prometheus:   { endpoint: 0.0.0.0:8889 }
  debug:        { verbosity: basic }
extensions:
  health_check: { endpoint: 0.0.0.0:13133 }
service:
  pipelines:
    traces:  [otlp] → [memory_limiter, batch]
             → [otlp/jaeger, spanmetrics]
    metrics: [otlp, spanmetrics] → [prometheus]
    logs:    [otlp] → [debug]
```

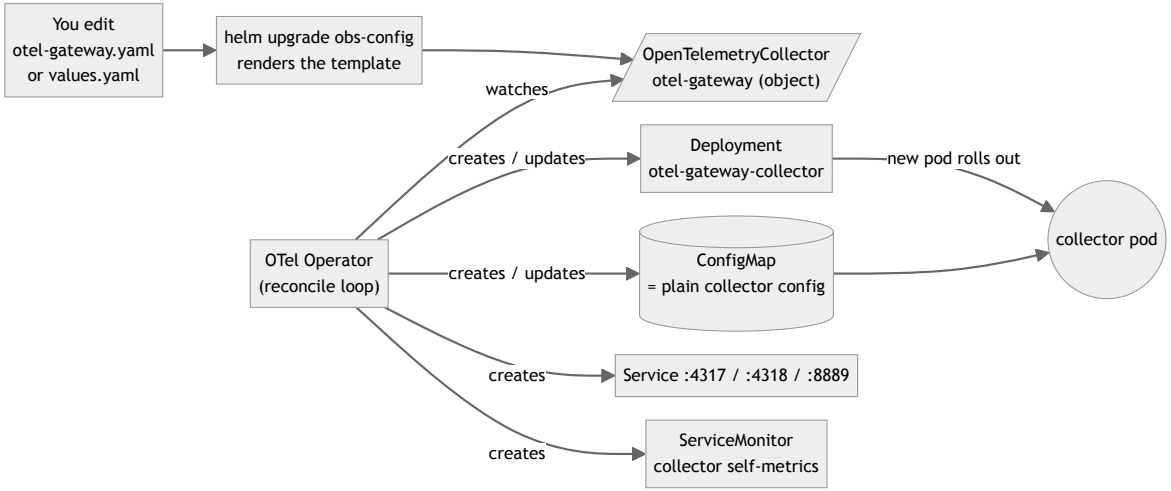
Kubernetes — templates/otel-gateway.yaml (object)

```
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector      # read by the OTel Operator
metadata: { name: otel-gateway }
spec:
  mode: deployment
  env: [NEW_RELIC_LICENSE_KEY ← Secret]   # if newrelic.enabled
  config:
    receivers: { otlp: grpc :4317, http :4318 }
    processors:
      memory_limiter: { limit_percentage: 80 }
      k8s_attributes: { deployment, pod, node, namespace }
      filter/unnamed: { drop unknown_service }
      batch: { timeout: 5s }
      resource/cluster, cumulative_to_delta   # New Relic
    connectors: { span_metrics: { exemplars: on } }
    exporters:
      otlp_grpc/jaeger: jaeger.observability:4317
      prometheus: 0.0.0.0:8889
      otlp_http/newrelic: otlp.eu01.nr-data.net
    service.pipelines:
      traces, metrics, traces/newrelic, metrics/newrelic
# + ServiceAccount, ClusterRole (k8s_attributes),
#   ServiceMonitor for :8889 — in the same file
```

DIFFERENCE	WHY
Wrapped in `OpenTelemetryCollector`	The Operator creates and updates the Deployment / Service / ConfigMap; no manual mount or restart.
`k8s_attributes` processor	Every span gets deployment, pod, node and namespace — context Docker didn't have. Needs read access to the Kubernetes API (ClusterRole in the same file).
Memory limit in % instead of MiB	Follows the pod's memory limit (512 Mi) automatically.
No `logs` pipeline / `debug` exporter	Logs go Alloy → Loki directly; the collector handles traces and metrics only.
Component names (`otlp_grpc` , `otlp_http` , `span_metrics` , `k8s_attributes`)	Collector 0.160 renamed components; old names still work but log deprecation warnings.
No `health_check` extension block	The Operator configures health and its own monitoring (`observability.metrics.enableMetrics`).
New Relic inside the same file, behind a switch	Docker used a separate overlay file; Helm's `{{ if .Values.newrelic.enabled }}` keeps one source of truth.

From a file to a running collector

In Docker, Compose mounts the file and starts the container. In Kubernetes, Helm renders the template into an object and the Operator does the rest.



See the live config

```
# the object Helm created
kubectl -n observability get opentelemetrycollector otel-gateway -o yaml

# the plain config the Operator generated (same format as Docker's file)
kubectl -n observability get configmap \
  -l app.kubernetes.io/name=otel-gateway-collector -o yaml

# the running pod and its logs
kubectl -n observability get pods -l app.kubernetes.io/name=otel-gateway-collector
kubectl -n observability logs deploy/otel-gateway-collector --tail=50
```

Change the collector

```
# 1. edit the template or a switch
#    charts/observability-config/templates/otel-gateway.yaml
#    charts/observability-config/values.yaml   (newrelic.enabled …)

# 2. apply — the Operator rolls out a new collector pod
helm upgrade obs-config charts/observability-config -n observability

# Docker equivalent, for comparison
docker compose up -d otel-collector
```

Same idea for Prometheus: instead of editing `prometheus.yml`, add a ServiceMonitor; the Prometheus Operator updates the scrape config. Same for Grafana: add a dashboard JSON; the sidecar loads it.